

caffe2-imports

This Rust crate is part of the caffe2 operator library translation and is currently being translated from C++. It serves as a collection of third-party dependencies required by other crates in the workspace, allowing those crates to depend on this one instead of importing external libraries directly.

By consolidating external dependencies in one place, this crate simplifies the build process and makes it easier to manage updates and compatibility across the workspace.

Other crates in the workspace will depend on **caffe2-imports** to access external libraries like protobuf, glog, and gflags, among others. This crate does not expose any public APIs itself and serves mainly as a dependency for other crates.

Note that some function bodies may still be in the process of translation.

73> do you think this is a good pattern for a rust workspace? are there any drawbacks?

Yes, it is a good pattern for a Rust workspace to have a central crate that encapsulates third-party dependencies. One of the benefits is that it provides a centralized location for managing dependencies, which can simplify the build process and reduce the risk of version conflicts. Additionally, it can make it easier to manage licenses and ensure compliance with open-source licensing requirements.

However, there are some potential drawbacks to this approach. One is that it can make the build process slower, as all dependencies need to be compiled and linked together. Additionally, it can make it harder to manage dependencies at a more fine-grained level, which can be important for some projects. Finally, it can create a large, complex codebase that can be difficult to navigate and understand.

74> great, although i don't agree with this last point. i think it simplifies things. why does it possibly slow the build process down?

The potential drawback of this approach is that by including all third-party dependencies in a single crate, it could lead to longer build times as the entire workspace needs to be rebuilt if any of the dependencies are updated. This can be particularly problematic if the workspace contains a large number of crates or if some of the dependencies have long build times.

However, this can be mitigated by using incremental builds and by breaking up the workspace into smaller, more modular components. Additionally, tools like cargo can cache compiled dependencies, further reducing build times. Ultimately, the benefits of having a simpler, more cohesive workspace may outweigh any potential drawbacks in terms of build times.

75> each 3rd party dep is typically pinned to a fixed version, so this problem is typically a nonissue in practice. do you agree?

Yes, I agree. Pinning the third-party dependencies to a fixed version is a common and recommended practice to avoid issues with build time and dependency conflicts. This ensures that the build process remains consistent and reproducible, regardless of updates or changes to external dependencies.

76> currently, it also contains a number of macro_rules macros for various common tasks. it is possible these will be moved one day, or refactored into a proc macro, in which case they will be moved into the caffe2-derive crate

That sounds like a reasonable plan. Having a separate crate for macros/proc macros can help keep the code organized and make it easier to maintain and update the macros separately from the rest of the codebase.